Embeddings

Modulo 2 · Lezione 7



Trovare ≠ cercare parole

Da Modulo 2 · Lezioni 4-6 avete documenti **puliti**. Ma un cliente chiede: *«come **imposto la posizione** del piano?»* — e la FAQ dice *«come **regolo l'altezza**»*. La stessa richiesta, detta con altre parole.

Cerca la stringa

`"imposto la posizione"` dentro la FAQ → **0 risultati**. Sinonimi, parafrasi, altre lingue: per il keyword matching sono *stringhe diverse*, punto.

Cerca il significato

«impostare la posizione» e «regolare l'altezza» **dicono la stessa cosa** — e un embedding le mette *vicine*, anche quando il lessico cambia.

Stasera il testo diventa **numeri che catturano il significato**. È il primo mattone del blocco retrieval (Modulo 2 · Lezione 7 → Modulo 2 · Lezione 10).



Agenda

Quando c'è da lavorare, una **slide di switch** vi manda al posto giusto — finita l'attività, si torna qui.
🎤 = il docente scrive, voi replicate · ⌨️ = scrivete voi (docente in affiancamento)

	Blocco	Dove si lavora	Tempo
📊	Teoria: embedding, spazio semantico, cosine, modelli e costi	slide	40′
🎤	**Live coding: il modulo embeddings**	`working/embeddings.py`	40′
⌨️	**Esercizio: il corpus Lumen**	`working/esplora_corpus.py`	30′
📊	Wrap-up: reveal delle coppie + il toolkit	slide	10′

> La chiave è **quella di Modulo 2 · Lezioni 1-3**: nessuna registrazione nuova stasera.

Setup (venv, requirements, `.env`, download del modello locale con `scarica_modello.py`): passo-passo in `scripts/CONSEGNA.md` — si fa durante la teoria se non l'avete già. Tutti i comandi della serata si lanciano **dalla cartella** `scripts/`.



1. Cos’è un embedding



Da testo a vettore

Un embedding è il risultato di una chiamata: entra una frase, esce una **lista di numeri**. Questa è vera, fatta col modello di stasera:

```
>>> embed(["Quanto dura la garanzia?"])
[-0.017, 0.021, 0.022, 0.020, -0.054, -0.011, -0.009, 0.018, ...]
# 1536 numeri (text-embedding-3-small)
```

La definizione

Quei 1536 numeri sono una **posizione in uno spazio del significato**, calcolata da un modello addestrato su miliardi di frasi. Stessa frase → stesso vettore. Frase *simile* → vettore **vicino**.

> «Vicino» lo definiamo tra qualche slide: si misura con un **angolo**, non con una distanza.



Lo spazio semantico

Proiettiamo i testi Lumen su un piano — ogni punto è un **testo del corpus**, etichettato con la sua parola chiave. I temi si raggruppano da soli: nessuno ha detto al modello che «sostituzione» sta con «garanzia».



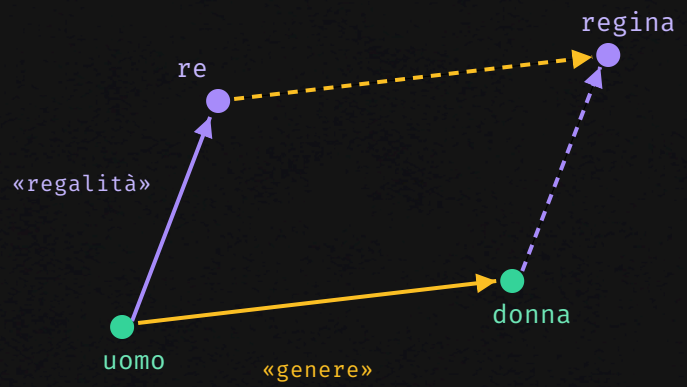
«sostituzione» e «garanzia» sono parole del tutto diverse — eppure finiscono nello stesso angolo. La vicinanza è di **significato**, non di parole.

Il disegno è una **proiezione**: lo spazio vero ha *centinaia di dimensioni*.

L’aritmetica del significato

Nel 2013 un esperimento rese famosi gli embedding (word2vec): coi vettori delle parole si può fare **aritmetica**. Si prende il vettore di `re`, si sottrae `uomo`, si somma `donna` — e il vettore più vicino al punto d'arrivo è `regina`.

`re - uomo + donna ≈ regina`



`donna - uomo` isola una **direzione** dello spazio: il genere. La stessa freccia, applicata a `re`, atterra vicino a `regina`; applicata ad `attore`, vicino ad `attrice`.

Il punto vero

Lo spazio codifica **tante informazioni insieme**, ognuna lungo una sua direzione: genere, singolare/plurale, tempo verbale, registro, lingua, tema. Ecco a cosa servono le **1536 dimensioni**.

> Nei modelli di *frasi* di stasera l'aritmetica non è così pulita — ma il principio regge: **le direzioni portano significato**, ed è quello che il retrieval sfrutta.

Dimensionalità

Quanti numeri per vettore lo decide **chi ha addestrato il modello**. Più dimensioni = più sfumature catturabili — e più byte da salvare e confrontare.

Modello	Dimensioni
`text-embedding-3-small`	**1536**
`text-embedding-3-large`	3072
`all-MiniLM-L6-v2` (locale)	384

Il parametro `dimensions`

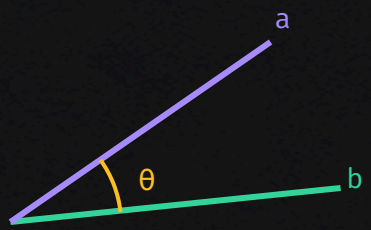
Ai modelli `text-embedding-3` puoi **chiedere un vettore più corto** (es. 256): si paga un po' in qualità, si risparmia parecchio in storage.



Cosine similarity: un angolo, non una distanza

Due vettori sono «vicini» se puntano nella **stessa direzione**. Si misura l'angolo tra loro:

$$\cos(\theta) = a \cdot b \,/\, (|a|\ |b|)$$



1.0
stesso significato

~0
scollegati

-1.0
opposti

> Coi modelli veri i valori tipici cadono tra **~0.2** (nulla a che fare) e **~0.9** (quasi identici). E niente librerie: è un **prodotto scalare**, lo scriviamo a mano tra poco.

Cosa vede (e cosa non vede)

Due proprietà da conoscere prima di costruirci sopra — le misuriamo **coi numeri** stasera stessa.

Vede: il TEMA

Sinonimi, parafrasi, giri di parole — e nei modelli multilingua persino **lingue diverse**: «adjust the desk height» finisce vicino a «regolare l'altezza». *Quanto* dipende dal modello: lo misurerete voi.

Non vede: la VERITÀ

«La **consiglio**» e «**NON** la consiglio» parlano dello *stesso tema* — e l'embedding le mette **vicine**. Non è un detector di verità né di sentimento.

Chi costruisce retrieval **deve** saperlo: a Modulo 2 · Lezione 8 ci sbatterete contro.



2. I modelli, e quanto costano



Il panorama dei modelli

Modello	Dove gira	Dim	Prezzo / 1M token	Note
`text-embedding-3-small`	API OpenAI	1536	$0.02	il default di stasera
`text-embedding-3-large`	API OpenAI	3072	$0.13	più qualità, 6× il prezzo
Cohere `embed-v4`	API Cohere	—	a consumo	stesso pattern, altro fornitore
`all-MiniLM-L6-v2`	**la vostra macchina**	384	**gratis**	88 MB, gira in CPU

Stasera li usiamo entrambi

API **e** locale, dietro la **stessa firma**: `embed(texts, backend="openai" | "local")`. Cambiare fornitore = cambiare una stringa.



Quanto costa davvero

Il corpus di stasera, embeddato per intero con `text-embedding-3-small`:

48	2.767	$0.000055
testi (tutto il corpus)	token	≈ mezzo centesimo ogni 100 giri

Per confronto: **una sola risposta** di chat di Modulo 2 · Lezione 1 costa ordini di grandezza in più. Embeddare è la parte *economica* della pipeline.

Economico ≠ gratis

A scala (milioni di testi) il conto torna a esistere — e ri-embeddare lo **stesso testo** è buttare soldi. Per questo il corpus embeddato stasera si **salva su disco**: si embedda una volta, si riusa. Mentalità Modulo 2 · Lezione 2.



Il trade-off: costo · latenza · privacy

Non c'è un vincitore: c'è il modello giusto per il **vincolo** giusto. Tempi misurati sui nostri 48 testi.

	API (`openai` **)**	**Locale (** `local` **)**
Costo	$0.02 / 1M token	zero
Latenza	1.5s (48 testi, via rete)	0.28s di encode (dopo ~6s di load)
Privacy	i testi **viaggiano** verso il servizio	i testi **restano in casa**
Qualità (ITA/multilingua)	alta	modesta — lo misurerete voi

Criterio pratico

Dati sensibili che non possono uscire → **locale**. Qualità multilingua e zero infrastruttura → **API**.

> Terzo criterio spesso dimenticato: la **stabilità** — se il fornitore spegne il modello, i vettori vecchi *non si confrontano* coi nuovi. Va ri-embeddato tutto.



🎤 Live coding — il modulo embeddings

🎤 **LIVE CODING — si guarda e si replica**

`working/embeddings.py` · 40 minuti

si torna alle slide per il debrief

TODO 1

`cosine_similarity`

a mano in numpy: prodotto scalare / norme

TODO 2

i due backend

`embed_openai` (col costo stampato a ogni chiamata) · `embed_local`

TODO 3

`embed()`

un if sui backend, `ValueError` sul resto

Il modello locale

È già sceso col setup (`scarica_modello.py` , 88 MB). Chi l'ha saltato lo lancia **ora**: `python scaffolding/scarica_modello.py`

Fatto quando

La demo in fondo al file gira sui **due backend** — A~B alta, A~C bassa: `python working/embeddings.py`



📊 Al rientro — debrief: openai e local a confronto

La demo appena vista, in numeri. Tre frasi: **A** «la scrivania si alza col doppio motore» · **B** «il piano motorizzato sale e scende» · **C** «il reso entro 30 giorni».

Coppia	`openai`	`local` (MiniLM)	Lettura
A ~ B — stesso senso, parole diverse	**0.650**	**0.668**	entrambi la vedono
A ~ C — temi diversi	**0.261**	**0.564**	qui si separano

I valori assoluti NON si confrontano tra modelli
0.65 di `openai` e 0.67 di MiniLM **non sono** «la stessa similarità». Conta l'**ordinamento dentro** un modello.

Sull'italiano il locale **separa peggio**: 0.668 vs 0.564 è un margine sottile; `openai` tiene le distanze (0.650 vs 0.261).

> Adesso i vettori li fate **voi**, sul corpus vero di Lumen.



⌨ Esercizio — il corpus Lumen

⌨ **TOCCA A VOI — docente in affiancamento**

`working/esplora_corpus.py` · 30 minuti

si torna alle slide per il reveal

`TODO 4 · indice_per_id`

Il **ponte**: le coppie parlano per *id*, i vettori per *riga* — costruite il dizionario id → indice.

`TODO 5 · misura_coppie`

Le **6 coppie assegnate** (4 normali + 2 **trappole**): stampate la tabella delle similarità, via TODO 4.

`carica_corpus()` e `salva()` sono **già scritte**: il giro completo lascia `corpus_embeddings.npz` + `corpus_meta.json` in `working/`.

Fatto quando

Tabella stampata + i 2 file in `working/`.

`Stretch (chi finisce prima)`

`BACKEND = "local"` e rilancio (gratis), poi richiudete con `"openai"`: ogni giro sovrascrive i 2 file.



📊 Al rientro — le 6 coppie, due backend

Colonna `openai`: il vostro giro. Colonna `local`: lo stretch — chi non l'ha fatto legge i numeri del docente (versioni diverse del modello spostano i decimali, l'**ordinamento** resta).

Coppia	`openai`	`local`	
garanzia ↔ ticket garanzia	0.516	0.521	condividono le parole esatte («garanzia», «SRV-03»)
«regolo l'altezza» ↔ «imposto la posizione»	**0.637**	**0.662**	stessa richiesta, altre parole — e batte la coppia qui sopra
spedizione ↔ estetica del bambù	0.284	0.417	il controllo: temi diversi
«la consiglio» ↔ «NON la consiglio»	**0.867**	**0.877**	🪤 la coppia PIÙ simile di tutte
stessa FAQ, italiano ↔ inglese	0.650	**0.193**	🪤 il locale crolla sul cross-lingua
descrizione LD-200 ↔ recensione LD-200	0.678	0.642	stesso prodotto, media

Trappola 1 confermata: il tema vince sulla verità — recensione entusiasta e stroncatura quasi gemelle (0.867).

Trappola 2 confermata: il multilingua non è scontato — 0.650 vs **0.193**. Il modello si sceglie *coi numeri*.



Cosa portate a casa

🧰

`embeddings.py` → `aikit/`

cosine a mano + 2 backend dietro una firma. Da stasera vive nel **toolkit**: si importa, non si riscrive. Tutti, ora:

```
cp working/embeddings.py
scaffolding/aikit/embeddings.py
```

📦

`corpus_embeddings.npz`

il corpus Lumen **vettorizzato** (+ `corpus_meta.json`), in `working/`. **Non cancellatelo**: nella prossima lezione si riapre.

🧠

Le tre regole

significato ≠ parole · tema ≠ verità · il modello è un **trade-off** (costo / latenza / privacy), si sceglie coi numeri.

Modulo 2 · Lezione 8: il corpus si interroga

Una **query** → embed → confronto con tutti i 48 vettori → i **5 testi più vicini**. In numpy puro, senza database: la vostra prima *ricerca semantica*.



Grazie

Prossima lezione — Modulo 2 · Lezione 8 · Semantic Search

